루프 엔지니어링 — 실습 가이드

명세가 루프를 설계한다.

> 목적을 이루는 설계 SPEC을 **조건 7가지**로 나눈다.
> 조건 하나가 **STEP 하나**가 되고, **앞 조건이 서야 뒤 조건이 선다.**
> STEP 7개에서 나온 **결정 21개**가 **웹페이지 하나**를 만드는 루프 셋(A·B·C)이 된다.

이 실습의 정체성

문제정의 기반 질문에 답하면,
그 답이 **명세를 생성하고,**
생성된 명세가 **Goal(Intent)을 이루는 루프를 만든다.**

내가 하는 일

- STEP마다 **A / B / C** 고르기 — 전부 **21개**
- 그중 **9개는 숫자**만 고른다
- 마지막에 **브라우저로 눈 확인**

킷이 이미 해 둔 일

- 화면 **이름표**(`data-*`) 고정
- **판정기 3개 + 최종 스위치**(`verify_all.py`) 제공 · **보호 파일 28개** 잠금
- **실패 라우팅 7종** 고정
- **정성 규율 10가지 · 3회** 고정

```
loop_starter_kit/    ← 오늘 다루는 폴더
  USER_REQUEST.md    내 취향 원문. 출발점이자 정성 검사의 기준
  specs/             STEP 1~7 명세. 여기서 21개에 답한다
  MASTER_SPEC.md     결정이 모이는 곳. 검사기는 여기서만 숫자를 읽는다
  LOOP_CONTRACT.md   실행 절차   scripts/ 잠긴 검사기   templates/ state/ evidence/ output/
```

주제는 여행 정보 웹페이지지만, 배우는 것은 **루프 설계**다.

루프가 성립하려면 — 조건 7가지

> 루프 : **Act → Observe → Reason → Repeat** 이 네 동작이 실제로 **닫히려면** 일곱 가지가 먼저 정해져야 한다.

#	조건	정해야 하는 것	안 정하면
①	**시작 상태**	루프가 무엇을 재료로 도는가	바퀴마다 다른 것을 만든다
②	**관측 표면**	기계가 무엇을 보고 판단하는가	"된 것 같다"로 끝난다
③	**충족 기준**	무엇이 몇 개면 됐다고 하는가	언제 멈출지 모른다
④	**행동 경계**	한 바퀴가 무엇을 손대는가	실패해도 원인을 못 찾는다
⑤	**판정 독립**	누가 채점하는가	기준을 낮춰서 통과한다
⑥	**반복 규칙**	실패하면 다음에 무엇을 하는가	같은 실패를 반복한다
⑦	**종료 선언**	무엇이 되면 끝인가	끝나지 않는다

> 오른쪽 칸은 전부 **실제로 겪는 실패**다. 조건을 빼먹거나 엉성하게 설계하면 루프는 돌긴 도는데 **퀄리티가 안 나온다.**

조건 7개가 곧 STEP 7개, 그리고 결정 21개

조건	STEP	이 STEP의 명세가 하는 일	내가 고를 결정
① 시작 상태	**1** 입력과 취향	국가·일수·스타일을 **루프의 고정 입력**으로 박는다	D-여행입력 D-취향분류 D-도시선정기준
② 관측 표면	**2** 페이지 구성안	세부 구현 사항에 **이름표**를 붙여 기계가 보게 한다	D-페이지구성 D-디자인요구 D-취향설명분량
③ 충족 기준	**3** 정보량 카운트	"충분하다"를 **세는 숫자**로 바꾼다	D-도시수 D-명소수 D-소개근거수 D-일정근거수
④ 행동 경계	**4** 경계·작업·안전	손댈 **작업**과 못 건드릴 **파일**을 자른다	D-작업목록 D-수정금지파일 D-사람확인지점
⑤ 판정 독립	**5** 테스트와 검증	검사기를 **루프 밖**에 두고 깐깐함만 고른다	D-정보량테스트 D-페이지테스트 D-최종검증명령
⑥ 반복 규칙	**6** 반복 방법	실패 유형을 **다음 행동**으로 바꾸고 상한을 건다	D-정보루프반복 D-페이지루프반복 D-무진전기준
⑦ 종료 선언	**7** 실행 계약	무엇이 되면 끝인지를 **문장 하나**로 확정한다	D-성공조건 D-최대반복

이 21개가 전부다. 루프 C(정성)는 **킷이 전부 고정**했다 — 조건이 늘어난 것이 아니라 같은 조건 7가지가 **한 겹 더 깊어진 것**이다.

순서는 **의존 관계**다. 어느 칸에 담을지(②)를 모르면 몇 개가 충분한지(③)도 못 정하고, 무엇을 손대는지(④)를 모르면 누가 채점하는지(⑤)도 못 정한다.

조건이 겨냥하는 결과물 — 파일 하나, 기능 셋

기능	무엇이 보이나	어떻게 움직이나
1 국가 소개	좌측 상단 국기 · 우측 상단 대표 사진 매력 · 역사 · 문화 · 국가 상태 · 여행 정보	제목을 누르면 **아코디언**이 펼쳐짐
2 도시와 관광지	도시 카드 → 관광지 목록 → 관광지 상세	도시를 누르면 **우측 툴바**가 열림 **뒤로 가기** 있음
3 도시별 일정	**도시마다 여행 일수 전체**를 시간 단위로 짠 계획	도시 제목을 누르면 **아코디언**

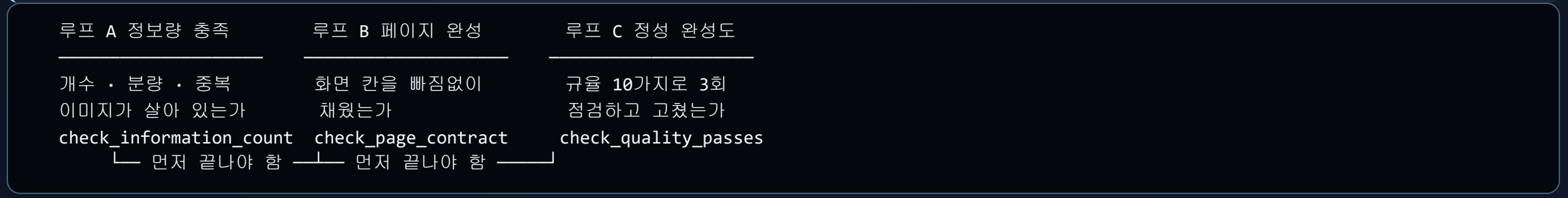
> 기능 2에는 **"왜 이 관광지를 넣었고 무엇을 뺐는지"**가 `D-취 향 설 명 분 량` 만큼 들어간다. **"A에는 사진 스팟 xxx가 있다. B의 박물관 yyy는 취향이 아니라 뺐다."**

> 기능 3은 **"며칠을 어떻게 돌까"가 아니라 "어느 도시로 갈까"**를 돕는다. 그래서 **도시마다 여행 일수 전체**가 각각 들어간다 — 셋을 나란히 놓고 하나를 고르도록.

> 최종 산출물은 `output/travel_site.html` **파일 하나**다. CSS·JS도 그 안에 넣는다.

충족 기준이 셋이라 — 루프도 셋

정보가 충분한가 · **화면이 다 찼는가** · **읽을 만한가**는 **서로 다른 기준**이다. 기준이 셋이니 **판정도 셋, 루프도 셋**.



왜 순서를 강제하나? 없는 정보로 화면을 만들면 **껍데기**만 나온다. 껍데기를 놓고 정성을 따지면 **3회를 거기서 다 쓴다.**

각 루프가 끝나면 **되물음**이 뜬다. "이만큼 채웠는데 계속할까, 넘어갈까?"

루프 C만 모양이 다르다. 앞 둘은 **검사기가 통과하면** 끝나지만, 루프 C는 **정해진 3회**를 반드시 돌아야 끝난다.

STEP 문서는 전부 같은 8구역

구역

1	이 STEP에서 정하는 것	읽기
2	앞에서 물려받은 결정	확인
3	이미 정해 둔 것	훑기
4	내가 결정할 질문	고민 ★
5	내 답 적기	적기
6	Claude가 정리한 결과	확인
7	끝나면 무엇이 진행되나	확인 ★
8	완료 확인과 다음 명령	체크

★ 둘만 챙기면 된다. **4**에서만 고민하고(질문 문장은 안 바뀐다), **7**이 다음에 칠 명령을 알려 준다.

진행하는 법 — STEP 1~7 동일

0 STEP 문서의 읽기 1~3을 훑는다 (개념만)
1 읽기 4에서 무엇을 결정해야 하는지 읽는다
2 "STEP N 진행해줘" 라고 말한다
3 Claude가 선택지 A/B/C와 권장값으로
나에게 역으로 질문한다
4 답하면 명세·결정 장부·뒤 STEP이 갱신된다

시작 명령

```
cd loop_starter_kit
python scripts/check_loop_readiness.py
claude
```

그다음 Claude Code 안에서 → **STEP 1 진행해줘**

처음 실행하면 **NOT_READY (exit 2)**가 뜬다. **오류가 아니다.** 아직 아무 질문에도 답하지 않았기 때문이다.

STEP 1 — 조건① 시작 상태

루프가 무엇을 재료로 도는가 — 이게 흔들리면 바퀴마다 다른 것을 만든다

명세가 박아 두는 것

```
국가          어느 나라
여행 일수     며칠
여행 스타일   뭐가 좋고 뭐가 싫은가
```

결정	A	B	C
`D-여행입력`	**베트남·5일**	일본·4일	직접
`D-취향분류`	2갈래	**3갈래**	3갈래+추가
`D-도시선정기준`	1등 유명도 2등 취향	**1등 취향** **2등 치안·교통**	1등 취향 2등 유명도 3등 교통

권장 **A · B · B** — 유명도는 **내 취향과 무관**하다.

스타일은 말하듯 쓴다

```
Preferred
  대표 관광지 다 가 보고 싶어.
  액티비티 있으면 무조건 넣어 줘.

Not preferred
  아 나는 박물관은 진짜 싫어.

Detailed preference
  애들 데리고 가는 가족 여행이야.
  인생 사진 핫플은 꼭 챙겨 줘.
```

결과물에 이렇게 나온다 — **"다낭은 액티비티가 많고 교통이 좋아 골랐다."**

원문은 `USER_REQUEST.md`에 **내 말투 그대로** 들어간다 — 루프 C가 "설명이 **원문 표현과 이어지는가**"를 이 파일로 본다. **출처 등급은 따지지 않는다.**

STEP 2 — 조건② 관측 표면

기계가 무엇을 보고 판단하는가 — 볼 수 없으면 판정도 없다

```
STEP 2 에서 "도시를 누르면 우측 툴바가 열린다" 라고 적으면
   ↓
검사기가 "우측 툴바가 있는가 / 누르는 연결이 있는가" 를 실제로 본다
   ↓
없으면 FAIL → 루프가 다시 돈다
```

결정	A	B	C	권장
D-페이지구성	기능1 아코디언 · 기능2 툴바 · 기능3 아코디언	기능1 탭 · 기능2 툴바 · 기능3 아코디언	셋 다 아코디언	**A**
D-디자인요구	밝은 배경 · 파랑 강조	어두운 배경 · 청록 강조	직접 지정	**A**
D-취향설명분량	10줄	15줄	20줄	**B**

```
아코디언    제목을 누르면 아래로 펼쳐지고, 다시 누르면 접힌다
우측 툴바   도시를 누르면 오른쪽에서 서랍처럼 밀려 나온다
탭          위에 단추가 나란히 있고, 누르면 아래 내용만 바뀐다
```

여기서 대충 적으면 대충 만들어도 통과한다. 화면 그림이 곧 시험 문제라 이 STEP이 결과물 품질을 결정한다. **줄 수만 정하면 같은 문장을 복사해 채운다.** 그래서 킷이 줄당 글자 수와 고유 줄 비율을 함께 건다 — 이건 고르지 않는다.

STEP 2 — 이름표는 고정, 배치만 고른다

세부 구현 목표 1 — 웹 페이지는 어떤 형태여야 하는가?

```
기능1  <section id="feature1">
       <img data-role="flag">                              국기 (좌측 상단)
       <img data-role="hero">                              대표 사진 (우측 상단)
       <button class="acc-header" data-acc="feature1" data-key="매력">  제목
       <div    class="acc-panel"  data-acc="feature1" data-key="매력">  펼쳐지는 내용
기능2  <section id="feature2">
       <button data-city="다낭">              도시 단추
       <aside  id="city-drawer">              우측 툴바
       <div    data-city-panel="다낭">        툴바 안 도시 칸
       <button data-spot="미케 비치">          관광지 단추
       <img    data-spot-image="미케 비치">    관광지 사진
       <div    data-spot-detail="미케 비치">   관광지 상세
       <div    data-preference-note="다낭">    취향 설명 (p 줄로 씀)
       <button data-back>                     뒤로 가기
기능3  <section id="feature3">
       <button class="acc-header" data-acc="feature3" data-key="다낭">  도시 제목
       <div    class="acc-panel"  data-acc="feature3" data-key="다낭">  도시 칸
         <div data-day="1">                   1일차   ← 도시 칸 안에
           <li data-block="09:00-11:30">      시간 블록
       <a data-source href="...">             출처 링크 (권장 표기 — 검사기는 모은 출처 주소가 화면에 있는지만 본다)
```

검사기는 이 **이름표**로 칸을 찾는다 — 하나라도 다르면 못 찾는다 (`data-acc`가 빠지면 아코디언 검사가 통째로 실패). **이름표는 킷 고정값**, 전문은 `specs/02_PAGE_DESIGN.md`. (단 `data-source`는 권장 표기다. 검사기는 이 이름표가 아니라 모은 출처 주소 글자가 화면에 있는지를 본다.)

`data-day`**는 반드시 도시 칸 안에 둔다.** 화면 전체에서 세면 **한 도시에 몰아 넣은 일정도 통과**한다. 검사기는 도시 칸 안에서 **도시마다** 센다.

STEP 3 — 조건③ 충족 기준

세부 구현 목표 2 — 각각의 정보량은 얼마나?

무엇이 몇 개면 됐다고 하는가 — 숫자가 없으면 멈출 지점도 없다

> 환율은 오늘 하나뿐이다. **1건 들어오면 그게 곧 충족이다.**

특성	뜻	충족 판정	해당 항목
다건	여러 건 모아야 풍성해지는 것	목표 개수 + **서로 다른 출처 2곳**	매력 · 역사 · 문화 · **주의사항**
단건	사실이 하나뿐인 것	**1건이면 끝**	항공권가격 · 환율 · 교통개요
조건부	지금은 없을 수도 있는 것	**확인 표시만 있으면 통과**	국가상태
자산	있고 없고인 것	주소 + 대체 문구 + **열어 본 기록**	국기 · 대표 · 관광지 사진

```
정보 슬롯 8개 → 기능1 헤더 5개로 묶인다.  정해 두지 않으면 루프가 추측하고 칸을 빠뜨린다
  매력 / 역사 / 문화+주의사항 / 국가상태 / 항공권가격+환율+교통개요

내가 고르는 숫자 4개  [ ]가 권장
  D-도시수 2[3]5   D-명소수 [3]5 8   D-소개근거수 [2]3 5   D-일정근거수 [2]3 5
```

> 단건 · 조건부 · 자산은 **데이터 특성이 이미 답을 정해 놓았다.** 그래서 내가 정할 것은 **숫자 4개뿐**이다.

STEP 3 — 세기 쉬운 모양으로 저장한다

output/research_data.json

```
{
  "country_intro": {
    "매력":     { "items": [ {"text":"…", "source_url":"…"} ] },
    "환율":     { "items": [ … ] },            ← 단건. 1개면 끝
    "국가상태": { "checked": true, "items": [] } ← 조건부. 0개도 통과
  },
  "cities": [
    { "name": "다낭",
      "why_selected": "왜 이 도시를 골랐는가",
      "spots":  [ { "name":"미케 비치", "items":[…], "detail":"…",
                    "image":{ "url":"…", "alt":"…",
                              "verified":{"status":200, "content_type":"image/jpeg",
                                          "checked_at":"2026-09-05T15:02:11"} } } ],
      "preference_note": "취향 연결 설명 (줄 단위)",
      "itinerary": { "items":[…],
        "days":[ {"day":1,"blocks":[
          {"start":"09:00","end":"11:30","kind":"명소","spot":"미케 비치","why":"…"} ]} ] } } ]
}
```

저장 모양은 고정이라 고르지 않는다. 이렇게 나눈 이유는 **세기 쉬워서**다 — `cities[0].spots`의 길이가 곧 "다낭의 관광지 개수"이고, 그 숫자가 **루프 A의 종료 조건**이다. `verified`는 **세 칸이 다 차야** 통과한다 — `status` · `content_type` · `checked_at`.

계산 — 3도시 × 3명소 = 관광지 9곳. 사진 9장 + 국기·대표 2장 = **이미지 11장**. 일정 칸 = 3도시 × 여행 일수 (도시마다 전체 일정을 각각 짠다).

STEP 3 — 숫자를 채웠는데 읽을 것이 없다면

실제로 한 바퀴 돌려서 나온 결과물 — 검사는 전부 통과했다

세 가지가 동시에 일어났다

관광지 상세	하한 120자 → 130자에서 끊음 (지금 300자로 올림)
취향 설명	30줄 중 21줄이 중복 복붙
이미지	11장 전부 죽음 (404 10장 · 400 1장) · 그런데 SUCCESS

왜 그렇게 되나

```
하한이 목표가 된다
   기준을 넘기면 멈춘다

줄 수가 목표가 된다
   내용이 아니라 줄이 늘어난다

주소가 있으면 통과다
   살아 있는지는 아무도 안 본다
```

하한만 올리면 복제가 늘어난다. 그래서 셋을 **같이** 건다. 전부 `MASTER_SPEC`의 계약에 있고 **교육생은 고르지 않는다.**

분량 하한	근거 1건 60자 · 관광지 상세 300자 · 도시 선정 이유 120자 · 취향 설명 줄당 30자
중복 상한	취향 설명 고유 줄 90% · 두 글 사이 유사도 60%
자산 유효	verified 기록 필수 · upload.wikimedia.org 주소 금지 (thumb·원본 모두 — Special:FilePath 형태만)

STEP 3 — 이미지는 왜 전부 죽었나

주소를 조회로 얻지 않고 형식에 맞춰 지어냈다

```
모델이 쓴 주소   .../thumb/8/8c/Moraine_Lake_17092005.jpg/1280px-...   →  404
실제 주소        .../thumb/c/c5/Moraine_Lake_17092005.jpg/1280px-...   →  200  (실재하지만 계약상 금지 형태)
```

파일도 사진도 실재했다. 틀린 것은 `8/8c` 한 조각뿐이다. 그 두 글자는 **파일명 해시에서 나온 값**이라 알아낼 방법이 없어 **지어낸 것**이다.

그래서 계약이 두 가지를 건다

```
1  만들 수 없는 주소 형태를 금지
2  열어 본 기록(verified)을 요구

권장 형태 — 파일명만 있으면 된다
commons.wikimedia.org/wiki/
   Special:FilePath/<파일명>?width=1024
```

검사기는 인터넷에 접속하지 않는다. 접속해서 확인한 기록을 본다. 그래야 판정이 빠르고, 인터넷이 없는 곳에서도 같은 결과가 나온다. 확인은 **모으는 쪽의 일**이고, 기록을 남기는 것이 계약이다.

STEP 4 — 조건④ 행동 경계

한 바퀴가 무엇을 손대는가 — 범위가 넓으면 실패해도 원인을 못 찾는다

루프	작업
A	`TASK-입력읽기` `TASK-국가정보` `TASK-도시선정` `TASK-명소수집` `TASK-일정수집`
B	`TASK-페이지생성` `TASK-기능점검`
C	`TASK-정성점검` × 3회 — **킷 고정**

첫 바퀴는 작업을 **차례로 한 번씩**, 다음 바퀴는 **지적된 칸의 작업만** 다시 한다. 지적 안 된 칸은 손대지 않는다.

결정	A	B	C
`D-작업목록`	**권장 7개 그대로**	5개로 합치기	직접 정의
`D-수정금지파일`	specs · scripts	A + templates CLAUDE.md	**B + MASTER_SPEC LOOP_CONTRACT**
`D-사람확인지점`	**루프 A·B·C 끝**	A + 부족할 때마다	맨 마지막 한 번

권장은 **A · C · A**.

`--lock`이 잠그는 **보호 파일 28개**는 킷이 못 박아 두었다 — `specs` `scripts` `templates` `CLAUDE.md` `MASTER_SPEC.md` `LOOP_CONTRACT.md`. `D-수정금지파일`은 **그 범위를 내가 명세에 선언하는 칸**이고, `specs` · `scripts`가 빠지면 준비도 검사가 막는다. **C를 고르면 선언과 실제 잠금이 정확히 일치한다** — AI가 기준을 낮춰 통과하는 길이 전부 막힌다.

STEP 5 — 조건⑤ 판정 독립

누가 채점하는가 — 시험지를 만든 사람이 시험을 보면 안 된다

실제로 있었던 일

```
AI가 만든 검사기  "칸이 3개 이상이면 통과"
AI가 만든 결과물  "칸 3개"
                → 통과!  그런데 명세는 원래 13칸을 요구했다.
```

검사는 통과했고, 결과물은 껍데기였다.

그래서 검사기는 루프 밖에 둔다. `scripts/**` 는 킷이 제공하고 `--lock` 으로 잠근다 — AI가 채점 기준을 낮출 길이 없다.

그리고 검사기는 숫자를 갖지 않는다. 글자 수 하한도 중복 상한도 정성 회차 수도 전부 `MASTER_SPEC.md` 에서 읽어 온다. 숫자를 바꾸려면 **명세를 바꾼다.** 스크립트는 건드리지 않는다.

STEP 5 — 검증 수준

D-정보량테스트 · 루프 A

```
A  개수만 센다
B  개수 + 서로 다른 출처
C  B + 분량 · 중복 · 이미지   ← 권장
```

A를 고르면 "좋음" 한 글자도 1건으로 센다.
C를 고르면 STEP 3에서 본 함정 셋이 막힌다.

D-페이지테스트 · 루프 B

```
A  칸이 있는가
B  A + 눌리는 연결이 있는가
C  B + 모은 내용이 실제로
       그 칸에 들어갔는가     ← 권장
```

A를 고르면 빈 칸만 있어도 통과한다.

D-최종검증명령은 A) `python scripts/verify_all.py`(권장) · B) 검사기를 따로 실행 · C) 직접 지정. 종료 스위치는 **한 줄**이어야 한다.

명령	무엇을 보나
`check_information_count.py`	루프 A — 개수 · 분량 · 중복 · 이미지 유효성
`check_page_contract.py`	루프 B — 화면 칸이 있고, 눌리고, 모은 내용이 들어갔는가
`check_quality_passes.py`	루프 C — 정성 점검을 규율대로 3회 돌았는가
`verify_all.py`	셋을 **순서대로** 돌린다. 최종 검증 명령

STEP 6 — 조건⑥ 반복 규칙

실패하면 다음에 무엇을 하는가 — 정해 두지 않으면 같은 실패를 반복한다

내가 정하는 상한

결정	A	B	C
`D-정보루프반복`	**3바퀴**	5	8
`D-페이지루프반복`	**3바퀴**	5	8
`D-무진전기준`	**같은 오류 2회**	3회	4회

권장은 **A · A · A**. 루프 C는 **3회 고정**이라 고르지 않는다.

```
오류 지문 = sha256(검사기 + 실패 유형 + 오류 첫 줄)
같은 지문이 반복 = 진전이 없다는 뜻
```

킷이 고정해 둔 라우팅

```
CODE_ERROR                → RETRY
INFORMATION_INSUFFICIENT → RESEARCH
SPEC_CONFLICT             → REVISE_SPEC
TEST_ERROR                → HITL
ENVIRONMENT_ERROR         → HITL
PERMISSION_ERROR          → HITL
UNKNOWN_ERROR             → STOP

HITL = 고치지 말고 사람에게 넘긴다
```

실패해도 처음으로 돌아가지 않는다. `RESEARCH`는 **부족으로 표시된 칸만** 다시 모은다.

STEP 6 — 루프 C는 왜 "3회"로 닫는가

통과선을 그을 수 없는 영역 — 그런데도 루프여야 한다

점수로 닫으면 무너진다

```
"품질 8점 이상이면 통과"
        ↓
글을 쓴 쪽이 자기 글에 점수를 준다
        ↓
조건 5 (판정 독립) 붕괴
```

STEP 5에서 본 **그 실패**와 같다.

그래서 회차로 닫는다

반복 횟수는 **관측 가능하고, 늘기만 하고,**
반드시 3에 닿는다.

종료 조건이 갖춰야 할 세 가지를
그대로 만족한다.

	루프 A · B의 상한	루프 C의 3회
뜻	**최대** — 여기까지만	**최소이자 최대** — 꼭 3번
통과	검사기 exit 0	3회를 규율대로 기록
정하는 사람	교육생	**킷 고정**

"세 번 다 '이상 없음'이라고 적고 끝내면 되지 않나?" — 그래서 판정기가 **품질이 아니라 절차**를 본다. 다음 장.

STEP 6 — 절차를 채점한다

회차마다 남기는 것

```
pass              회차 번호 1 · 2 · 3
focus             그 회차의 렌즈. 계약과 같아야 함
snapshot_before   파일 해시
rules[10]         10개 전부 OK 또는 VIOLATION
applied_fix[]     실제로 고친 것
snapshot_after    파일 해시
verify_after      exit 0 이어야 함
```

양식은 `templates/QUALITY_PASS.md`

판정기가 보는 것

```
본다  3회를 다 돌았는가
본다  10개를 전부 판정했는가
본다  위반이라 적은 것을 고쳤는가
본다  고쳤다면 해시가 바뀌었는가
본다  안 고쳤다면 그대로인가
본다  앞 회차 끝과 이번 회차 시작이
      같은 해시로 이어지는가
본다  루프 A·B가 여전히 통과하는가
본다  위반이면 어디(where)인지 적었는가
본다  3회 합쳐 고친 것이 1건 이상인가
      (0건이면 SHORT → 되물음: 다시 돈다 / 이대로 확정)
본다  마지막 회차 재검증을 직접 다시 돌린다

안 본다  글이 좋아졌는가
```

되돌아감 방지가 안전핀이다. 회차 끝마다 `verify_all.py --skip-quality`를 돌리고, `exit 0`이 아니면 **그 회차의 수정을 되돌린다.** 이게 없으면 **정성 개선이 정량 계약을 파괴하는 루프**가 된다.

STEP 6 — 정성 규율 10가지 (킷 고정)

회차마다 **10개를 전부** 판정하고, 그 회차의 **렌즈부터** 고친다

1회차 · 내용

- Q-반복금지
- Q-취향연결
- Q-뺀이유
- Q-상세삼요소

2회차 · 실행 가능성

- Q-일정현실성
- Q-근거일치
- Q-도시차별화
- Q-실용누락

3회차 · 독자 관점

- Q-동행반영
- Q-계약무결

+ 앞 회차가 되돌아가지 않았는지

같은 렌즈로 세 번 보면 2회차부터 "이상 없음"만 나온다. 그래서 렌즈를 나눈다. 전문은 `specs/QUALITY_RUBRIC.md`에 있고 **교육생은 고르지 않는다.**

루프 C가 잡는 것의 예 — 하루 블록이 **전부 명소이고 이동·식사가 없다**. 블록 수와 시간 형식은 맞으니 **정량 검사는 통과**한다. 사람은 즉시 안다.

되물음 — 상한에 걸려도 그냥 멈추지 않는다

부족이 남았을 때의 되물음이다 — 못 채운 칸 없이 통과했으면 [2]는 **"다음 루프로 넘어간다 ← 권장"**이고 `HALT_*`가 붙지 않는다

```
[되물음] 루프 A를 3바퀴 돌았지만 2칸을 못 채웠습니다.
  못 채운 것 : 후에 황제릉 사진 · 다낭 일정 근거 1건
  같은 오류가 2번 반복되었습니다

  고르세요
   [1] 한 바퀴만 더 돈다
   [2] 부족한 채로 루프 B로 넘어간다   ← 권장
   [3] 여기서 멈춘다
```

왜 [2]가 권장인가? 인터넷에 없는 정보는 몇 바퀴를 더 돌아도 없다. 작은 관광지는 쓸 만한 사진이 아예 없기도 하다. 남은 것을 완성하는 편이 낫다.

부족한 채 고른 [2]는 "성공"이 아니다. 못 채운 칸은 보고서에 그대로 적히고, 최종 검증은 **루프 A가 통과해야** 루프 B를 본다. 그래서 부족한 채 끝내면 상태는 `HALT_*`다. 그래도 **조용히 넘어가는 것**보다는 낫다.

STEP 7 — 조건⑦ 종료 선언

무엇이 되면 끝인가 — 선언하지 않으면 루프는 끝나지 않는다

결정 ID	A	B	C	권장
`D-성공조건`	검증 명령 exit 0	A + **사람이 브라우저로 확인**	B + 정보량 100%	**B**
`D-최대반복`	10	15	20	**A**

```
A   기계만 통과하면 끝 (정보량 + 페이지 + 정성 3회). 눈으로는 안 본다.
B   거기에 내가 브라우저로 한 번 본다.   사진이 그 관광지가 맞는지는 사람만 안다
C   B 와 같다 — exit 0 에 정보량 100%가 이미 들어 있다.
```

A만 골라도 정성 3회가 자동으로 포함된다. `verify_all.py`가 **루프 A·B·C를 모두** 보기 때문이다. 종료 스위치는 **하나**다. **exit 0 은 정보량 100%다.** 못 찾는 사진이 있으면 A·B 도 SUCCESS 가 아니라 `HALT_*`다.

STEP 7이 끝나면 **MASTER_SPEC.md 검사 계약의 `null` 이 전부 사라진다.** 하나라도 남으면 검사기는 **ERROR** — "덜 만들었다"가 아니라 **"명세가 안 끝났다"**는 뜻이다.

준비도 검사는 **세 곳**을 함께 본다 — `MASTER_SPEC.md` · `LOOP_CONTRACT.md` · `state/loop_state.json`. 그리고 **결정 장부 5번**에 내가 고른 21개가 만든 **루프 구조도**가 그려진다. `specs/00_DECISION_LOG.md`

한 STEP은 이렇게 흘러간다

예시 — STEP 3 진행 화면

```
나     : STEP 3 진행해줘

Claude : 이 STEP은 "충분하다"를 숫자로 바꿉니다.
         읽기 3은 이미 정해져 있으니 고민하지 않아도 됩니다.
         STEP 1에서 베트남 5일로 정하셨습니다.

         Q1. 관광 도시를 몇 개 고를까요?    A) 2   B) 3   C) 5    권장 B
         Q2. 도시 하나에 관광지는 몇 개?    A) 3   B) 5   C) 8    권장 A
             → 이 둘이 조사량 · 사진 장수 · 루프 시간을 정합니다.

나     : Q1은 B, Q2는 A로 할게요

Claude : D-도시수 = 3, D-명소수 = 3 으로 확정했습니다.
         관광지 총 개수 = 3 × 3 = 9곳
         필요한 이미지 = 9 + 2(국기·대표) = 11장
         갱신: specs/03, 결정 장부, PROJECT_MEMORY, specs/04, specs/05, specs/07
         검사: validate_specs.py --step 3  → exit 0
         코드는 만들어지지 않았습니다. 다음: "STEP 4 진행해줘"
```

질문은 **한 번에 2~3개씩** 온다. 남은 `D-소개근거수` `D-일정근거수`는 이어서 묻는다.

결정 하나가 뒤로 흐른다 — 그리고 바꾸려면

흐름

```
STEP 1  베트남 · 5일
   ▼
STEP 3  도시 3 × 명소 3 = 관광지 9곳
   ▼
MASTER_SPEC  count_contract
   { city_count:3, spots_per_city:3, days_per_city:5 }
   ▼
검사기  "도시 3개 / 관광지 9곳 / 도시마다 5일 있는가"
   ▼
STEP 7  성공 조건 문장에 그 숫자가 들어간다
```

바꾸려면

```
나 : D-명소수를 3에서 5로 바꾸고 싶어

Claude : 영향 범위입니다.
   관광지 총 개수    9 → 15
   필요한 이미지     11 → 17장
   루프 A 예상 시간  15분 → 30분
   재검토: specs/03 · specs/04 · specs/05 · specs/07
           결정 장부 · MASTER_SPEC
   진행할까요?
```

남기는 것: **이전 값 · 새 값 · 이유 · 시각**

> **MASTER_SPEC의 검사 계약까지 같이 고쳐야 한다.** 검사기는 거기서만 숫자를 읽으므로, 명세 본문만 고치면 **검사 기준은 그대로다.**

STEP 7 이후 — 여기서부터 코드가 생긴다

```
python scripts/check_loop_readiness.py         # READY 인가?
python scripts/check_loop_readiness.py --lock  # 보호 파일 28개 해시 잠금 ('잠금 확인됨'이 나오면 건너뜀)
```

```
루프를 실행해줘
```

```
생기는 것:  output/research_data.json   output/travel_site.html
           evidence/runs/<run_id>/           events.jsonl · test_results/
           evidence/runs/<run_id>/quality/   pass1 · pass2 · pass3.json

python scripts/export_graph_handoff.py   ← 루프가 끝난 뒤 (성공이든 중단이든)
           output/graph/  파일 7개 — 다음 강의로 넘길 인계본 (수집 전이면 6개 → 덮어쓰지 않음)
           작업 8개 → 루프 골격 14개 · 실패 라우팅 → 라우터 6개 · 되물음 → 게이트 2개
           검사기 뒤 진단 노드 2개 + 규칙 처방 가지 3개(노드 6개)까지 적힌다 → 노드 22개
```

`--lock`은 보호 파일 **28개를 해시로 기록**한다. 하나라도 바뀌면 **준비도 검사가 ERROR**를 내고 루프를 시작·재개할 수 없다. 조건⑤(판정 독립)를 기계로 강제하는 장치다.

어느 작업이 어느 순서로 도는지 헷갈리면 **결정 장부 5번**을 본다 — 5-1 큰 그림 · 5-1b 한 바퀴 안쪽 · 5-1c 루프 C · 5-2 실패 라우팅 · 5-3 내 결정이 조종하는 자리 · 5-4 그래프 대응.

루프 한 바퀴

```
Act      첫 바퀴는 작업을 차례로 · 다음 바퀴는 지적된 칸만  ← TASK-명소수집 (다음 바퀴)   조건 4
Observe  검사기를 한 번 돌리고 종료 코드를 읽는다           ← exit 2                    조건 2 · 5
Reason   실패 유형을 고른다                                  ← INFORMATION_INSUFFICIENT   조건 6
Repeat   정해진 다음 행동을 한다                             ← RESEARCH (부족한 칸만)     조건 6
```

실제 화면 (일부 줄임)

```
  검사 항목                                  목표      실제   판정
  ---------------------------------------------------------------
  기능1 매력                                  2       2   OK
  기능1 환율 (1건이면 충족)                        1       1   OK
  다낭/미케 비치 이미지                      살아 있는 이미지 1      부족   부족
  후에 · 취향 설명 고유 줄 비율                     90%     33%   부족
  기능3 후에 일정 일수                             5       2   부족

[부족] 6곳이 기준에 못 미칩니다.
  - 다낭/미케 비치 이미지: 목표 살아 있는 이미지 1 / 실제 부족
  - 다낭/미케 비치 이미지: verified 기록이 없습니다 (수집할 때 실제로 열어 보고 적습니다)
  - 후에 · 취향 설명 고유 줄 비율: 목표 90% / 실제 33%
  - 후에 취향 설명에 같은 문장이 9번 복사되어 있습니다 — <복사된 문장 앞 24자>
  - 기능3 후에 일정 일수: 목표 5 / 실제 2
  - 기능3 후에: [3, 4, 5]일차가 비어 있습니다
다음 행동: RESEARCH — 위에서 부족으로 표시된 곳만 다시 모읍니다.
          전부 다시 하지 않습니다.
```

멈추는 조건과 종료 코드

멈추는 상태 7가지

```
SUCCESS              성공 조건 충족
HALT_NO_PROGRESS     같은 오류 지문 반복
HALT_RESEARCH_GAP    정보를 끝내 못 채움
HALT_SPEC_CONFLICT   명세끼리 어긋남
HALT_PERMISSION      권한 없음 · 보호 파일
HALT_ENVIRONMENT     실행 환경 문제
HALT_NEEDS_HUMAN     사람 결정 필요
```

종료 코드 3가지

```
0  PASS     통과
1  ERROR    깨졌다 — 사람이 고쳐야 함
2  SHORT    아직 덜 됐다 — 정상, 계속 돈다
```

`check_loop_readiness` → `NOT_READY`는 **2**
`verify_all` → 아직 미충족도 **2**

1은 더 돌려도 해결되지 않는다. 검사기·명세·권한·환경이 원인이라
계약이 정한 다음 행동은 `HITL` — 사람에게 넘긴다.

> 좋은 루프는 **빨리 도는 루프**가 아니라 **멈출 줄 아는 루프**.

증거는 폴더에 남는다

```
evidence/runs/2026-09-05T14-20-05/
  run_manifest.json     언제 무엇을 왜 돌렸나
  events.jsonl          반복마다 한 줄
  test_results/         검사기 출력 원본
  changes/              바꾼 파일 목록과 해시
  quality/pass1~3.json  정성 회차마다 한 파일
  final_report.md       또는 failure_report.md (+ 사람 결정이 필요하면 reverse_question.md)
```

```
{"iteration":3,"loop":"A","task_id":"TASK-명소수집",
 "exit_code":2,"failure_type":"INFORMATION_INSUFFICIENT",
 "short_slots":["다낭/한강 용다리 이미지"],
 "error_hash":"sha256:ab12…","next_action":"RESEARCH"}
```

> 남기지 않는 것: **API 키 · 토큰 · 개인정보 · 모델의 비공개 사고 과정 전체**
> 반드시 남기는 것: **못 채운 칸의 목록**

누가 무엇을 판정하는가

무엇	예전	지금
글이 짧은가	사람	**코드** — 계약의 분량 하한
같은 말을 복사했는가	사람	**코드** — 고유 줄 비율 · 유사도
이미지가 열리는가	아무도 안 봄	**코드** — 열어 본 기록을 검사
일정이 도시마다 완결인가	사람	**코드** — 도시 칸 안에서 센다
일정이 현실적인가	사람	**루프 C** — 규율로 3회 점검
도시별 계획이 서로 다른가	사람	**루프 C**
사진이 그 관광지가 맞는가	사람	**사람** — 기계는 열리는지만 안다
환율·가격이 오늘도 맞는가	사람	**사람**
이 여행을 내가 하고 싶은가	사람	**사람**

기계가 **셀 수 있는 것**은 전부 기계로 옮겼다. 그래야 **루프 C의 3회**를 정말 사람이 아니면 못 보는 것에 쓸 수 있다.

흔한 실수 — "잘 된 것 같습니다"를 완료로 믿기 · **검사기를 고쳐서 통과시키기** · 지적되지 않은 칸까지 고치기 · **못 채운 칸을 조용히 넘기기** · **정성 점검을 "봤습니다"로만 적기**

정리

> 루프가 성립하는 **조건 7가지**에 답하면 명세가 완성되고,
> 그 명세가 **멈출 줄 아는 루프 셋(A·B·C)**을 만든다.

```
조건 7가지에 답한다              시작 · 관측 · 충족 · 경계 · 판정 · 반복 · 종료
      ▼
STEP 1~7 명세가 완성된다         결정 21개. 여기까지 코드는 생기지 않는다
      ▼
루프 A 정보량 → 루프 B 페이지     통과선으로 닫는다
      ▼
루프 C 정성 3회                  통과선이 없어 회차로 닫는다
      ▼
output/travel_site.html          파일 하나
```

셀 수 있는 것은 전부 계약으로. 분량 · 중복 · 이미지 · 일정 형식.

셀 수 없는 것은 회차로. 점수를 매기지 않고 **절차**를 채점한다.

다음 강의에서는 이 루프를 **그래프**로 옮긴다. **작업 8개(권장 7개 + 정성점검)가 루프 골격 14개**가 되고, 실패 라우팅이 간선이 된다. 검사기 바로 뒤 **진단 노드 2개**가 원인을 가려, 정해진 고칠 법이 있는 실패만 **규칙 처방 가지**(고치고 → 시험 → 검사기로 복귀)로 보낸다. 되물음은 **게이트 2개**가 되어 사람이 되돌아갈 곳을 고른다.
`export_graph_handoff.py`가 만든 `output/graph/` **7개**를 그대로 들고 간다 — **계약은 그대로, 실행 구조만** 바뀐다.